ByteMart Digital

THE COMPLETE GUIDE

Kafka

From Fundamentals to Professional Practice

A complete, example-driven guide to Apache Kafka — from topics, producers, and consumers to replication, delivery semantics, stream processing, integration, and operations.

30	200+	14	8
Chapters	Code Examples	Visual Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Kafka: The Complete Guide

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided “as is”.

Trademarks
Apache Kafka is a trademark of the Apache Software Foundation. All other product names, logos, and brands are the property of their respective owners and are used for identification purposes only. An independent educational work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. What Is Kafka? 4
2. Topics, Partitions & Offsets 7
3. Brokers & the Cluster 10
4. The Commit Log & Storage Model 13
5. Producing Messages 16
6. Keys, Partitioning & Ordering 19
7. Acknowledgements & Durability 22
8. Batching, Compression & Throughput 25
9. Consuming Messages 28
10. Consumer Groups & Rebalancing 31
11. Offsets & Commit Strategies 34
12. Consumer Delivery Semantics 37
13. Replication & the ISR 40
14. Leaders, Followers & Failover 43
15. Delivery Guarantees End to End 45
16. Idempotent & Transactional Producers 48
17. Serialization & Formats 51
18. The Schema Registry 54
19. Schema Evolution & Compatibility 57
20. Introduction to Kafka Streams 60
21. Stateless Operations 63
22. Stateful Operations & Windowing 66
23. KStreams, KTables & Joins 69
24. Kafka Connect 72
25. Connectors & Transformations 75
26. Kafka with Spring 78
27. Event-Driven Microservices 81
28. Monitoring & Performance 83
29. Security 86
30. Best Practices & Anti-Patterns 89

MODULE 1 · KAFKA FOUNDATIONS

CHAPTER 01

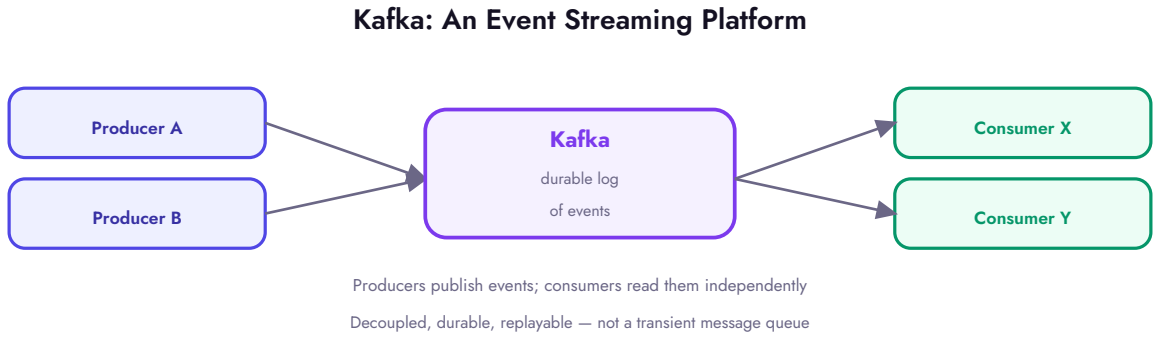
What Is Kafka?

An event streaming platform built on a distributed, durable, replayable commit log.

Apache Kafka is a distributed event streaming platform. At its core it is a durable, append-only log that many producers write to and many consumers read from independently. That simple idea — a replayable log rather than a transient queue — is what makes Kafka so powerful for modern data systems.

> **Kafka**
> A distributed event streaming platform that stores streams of records durably in ordered, partitioned logs, letting many producers and consumers exchange events at scale.

The core idea



Producers publish events to Kafka's durable log; consumers read them independently.

Not just a message queue

A traditional queue deletes a message once it is consumed. Kafka **retains** events for a configured period, so many independent consumers can read the same data, and any consumer can **replay** from the past. This changes what you can build.

	Traditional queue	Kafka
After read	message removed	retained
Consumers	compete for messages	each reads all
Replay	not possible	seek to any offset

What Kafka is used for

- **Event-driven microservices** — services communicate via events.
- **Data pipelines** — stream data between systems reliably.
- **Stream processing** — transform and aggregate events in real time.
- **Event sourcing** — the log is the source of truth.

© 2026 ByteMart Digital

- **Metrics and logs** — high-throughput ingestion at scale.

Why it scales

Kafka is **distributed** and **partitioned**: a stream is split across many machines, so throughput scales horizontally. It is also **replicated**, so it tolerates machine failures without losing data — the subjects of the chapters ahead.

> **A durable, replayable log**
> Kafka is an event streaming platform built on a distributed, durable commit log. Unlike a queue, it retains events for independent consumption and replay — the foundation for microservices, pipelines, and stream processing at scale.

© 2026 ByteMart Digital

MODULE 1 · KAFKA FOUNDATIONS

CHAPTER 02

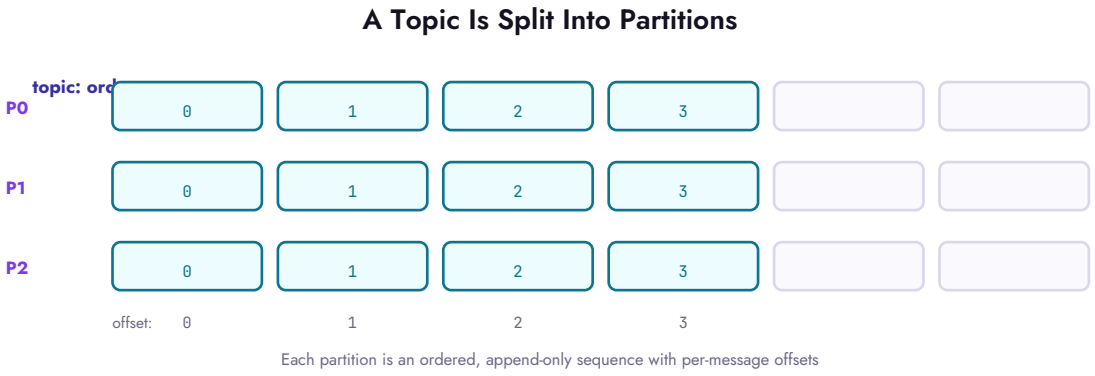
Topics, Partitions & Offsets

The core abstractions — how Kafka organises events into partitioned, ordered logs.

Kafka's data model rests on three concepts: **topics** group related events, **partitions** split a topic for scale and parallelism, and **offsets** give every record a position. Understanding these three is understanding how Kafka stores and orders data.

Topics and partitions

A **topic** is a named stream of events (like `orders`). Each topic is divided into **partitions** — independent, ordered logs. Partitioning is what lets a topic scale beyond one machine and be consumed in parallel.



A topic splits into partitions, each an ordered log with per-message offsets.

Offsets

Within a partition, every record has a monotonically increasing **offset** — its position in that log. Offsets are how consumers track what they have read and where to resume.

- A **topic** is a logical stream of events.
- A **partition** is an ordered, append-only log within a topic.
- An **offset** is a record's position within a partition.
- Order is guaranteed **within** a partition, not across partitions.

The key ordering rule

WATCH OUT

Kafka guarantees ordering only **within a single partition**, never across a topic. If two events must be processed in order, they must go to the same partition — which, as the next chapters show, means giving them the same key.

Choosing a partition count

More partitions mean more parallelism but more overhead. The partition count sets the maximum consumer parallelism for a topic, and is hard to reduce later — so choose it with growth in mind.

© 2026 ByteMart Digital

Topics, split into ordered partitions

A topic is a stream of events split into partitions — ordered logs indexed by offset. Partitions give Kafka its scale and parallelism, and because order holds only within a partition, partitioning drives everything about how Kafka behaves.

© 2026 ByteMart Digital

MODULE 1 · KAFKA FOUNDATIONS

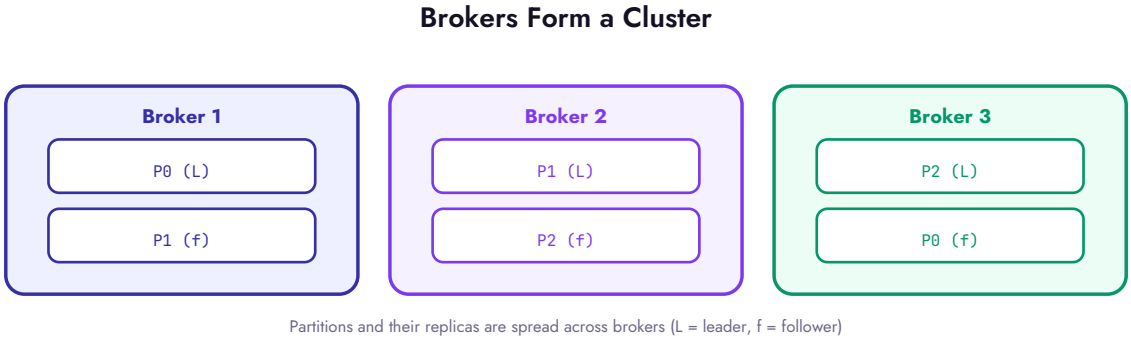
CHAPTER 03

Brokers & the Cluster

How Kafka servers form a cluster and distribute partitions for scale and resilience.

Kafka runs as a cluster of servers called **brokers**. Together they store the partitions of every topic, distribute the load, and replicate data for fault tolerance. The cluster is what turns the log abstraction into a scalable, resilient system.

Brokers and partition distribution



Brokers form a cluster; partitions and their replicas are spread across them.

Each **broker** hosts a subset of the partitions. Spreading partitions across brokers distributes both storage and the read/write load — add brokers, and the cluster's capacity grows.

Coordinating the cluster

The cluster needs consensus on metadata — which broker leads which partition, cluster membership, and configuration. Modern Kafka uses the built-in **KRaft** protocol for this; older versions relied on an external ZooKeeper ensemble.

	Older Kafka	Modern Kafka
Coordination	ZooKeeper	KRaft (built-in)
Metadata	external ensemble	internal quorum
Operations	two systems	one system

How the cluster provides resilience

- Partitions are **replicated** across multiple brokers.
- One broker is the **leader** for each partition; others are followers.
- If a broker fails, a follower takes over as leader.
- Clients discover the cluster via **bootstrap servers** and adapt automatically.

© 2026 ByteMart Digital

Bootstrap servers

Clients connect using a list of **bootstrap servers** — any brokers to start from. The client then learns the full cluster topology and connects to the right broker for each partition automatically.

A cluster of brokers

Kafka runs as a cluster of brokers that distribute partitions for scale and replicate them for resilience. Modern Kafka coordinates itself with KRaft, and clients bootstrap from any broker and discover the rest — the cluster makes the log fault-tolerant.

© 2026 ByteMart Digital

MODULE 1 · KAFKA FOUNDATIONS

CHAPTER 04

The Commit Log & Storage Model

The append-only log at Kafka's heart, and how retention makes events durable and replayable.

Everything in Kafka comes back to one data structure: the **append-only commit log**. Records are only ever appended, never modified, and reads seek by offset. This deceptively simple model is what gives Kafka its speed, durability, and replayability.

The append-only log



Records are only appended; reads seek by offset, and retention keeps data for replay.

New records go to the end of the partition log; existing records are immutable. This append-only design is extremely fast — sequential disk writes — and simple to reason about.

Retention, not deletion on read

Consuming a record does not remove it. Kafka keeps records according to a **retention policy** — by time (e.g. seven days) or size — independent of who has read them. This is what enables replay and multiple independent consumers.

Retention	Keeps data
time-based	for a set duration (e.g. 7 days)
size-based	up to a size limit per partition
compacted	the latest value per key, forever

Log compaction

A special retention mode, **log compaction**, keeps only the most recent record for each key — turning a topic into a durable, replayable snapshot of current state. It underlies changelog topics and stateful stream processing.

Why the log model wins

- **Fast** — sequential appends and offset-based reads.
- **Durable** — persisted to disk and replicated.

© 2026 ByteMart Digital

- **Replayable** — any consumer can re-read from any offset.
- **Multi-subscriber** — many consumers, one copy of the data.

It all comes back to the log

Kafka stores every partition as an immutable, append-only commit log, retained by time or size rather than deleted on read. This model delivers speed, durability, and replay — and compaction turns a topic into a snapshot of latest state.

© 2026 ByteMart Digital

MODULE 2 · PRODUCERS

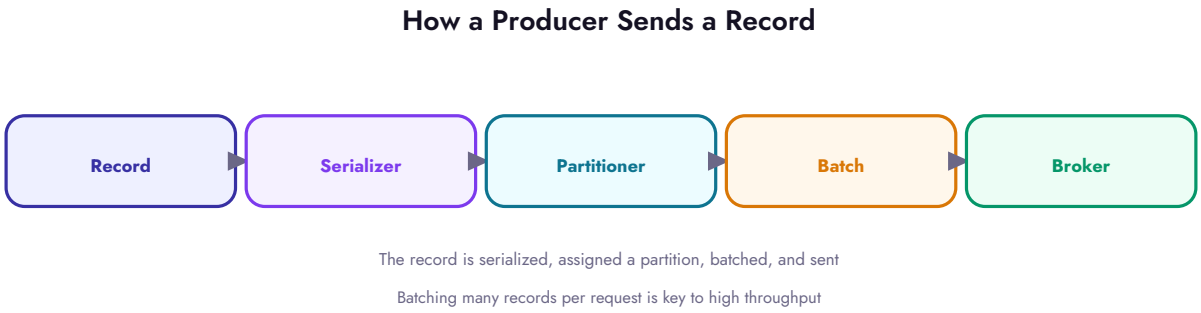
CHAPTER 05

Producing Messages

How a producer sends records to Kafka — the send path, keys, and asynchronous delivery.

A **producer** writes events to Kafka. It serializes each record, decides which partition it belongs to, batches it with others, and sends it to the right broker. Understanding this path explains both the producer's performance and its configuration.

The producer send path



A record is serialized, assigned a partition, batched, and sent to a broker.

A basic producer

```java
Properties props = new Properties();
props.put("bootstrap.servers", "broker:9092");
props.put("key.serializer", StringSerializer.class.getName());
props.put("value.serializer", StringSerializer.class.getName());

try (var producer = new KafkaProducer<String, String>(props)) {
    producer.send(new ProducerRecord<>("orders", "user-42", json));
}
```

Asynchronous by default

`send()` is asynchronous — it returns immediately, queuing the record for delivery. To know the outcome, use the returned future or a callback.

```java
producer.send(record, (metadata, exception) -> {
    if (exception != null) log.error("send failed", exception);
    else log.info("sent to partition {} offset {}",
                  metadata.partition(), metadata.offset());
});
```

Records carry a key and value

- The **value** is the event payload.
- The optional **key** determines the partition (next chapter).
- Both are serialized by configured serializers.
- Records may also carry **headers** for metadata.

TIP

Reuse a single producer instance across your application — it is thread-safe and maintains connections and batching buffers. Creating a producer per message destroys throughput and wastes resources.

Serialize, partition, batch, send

A producer serializes each record, assigns it a partition, batches it, and sends asynchronously to a broker. Reuse one thread-safe producer, use callbacks to confirm delivery, and remember the key drives partitioning.

© 2026 ByteMart Digital

MODULE 2 · PRODUCERS

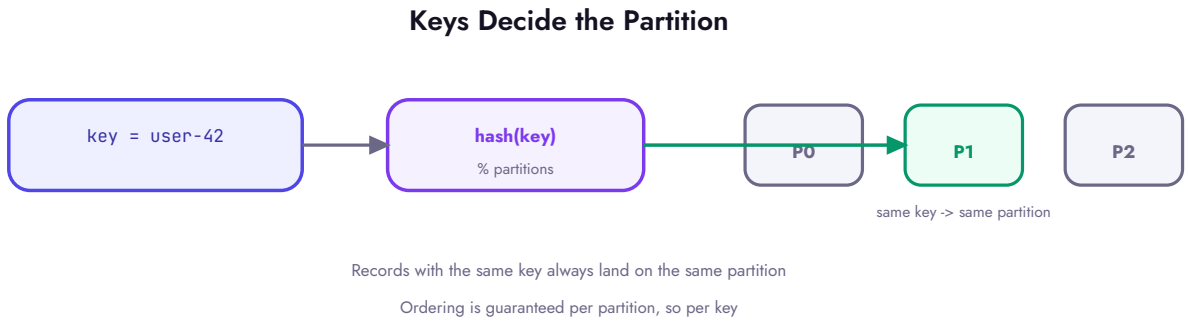
CHAPTER 06

Keys, Partitioning & Ordering

How the record key selects a partition, and why that determines ordering.

The record **key** is one of Kafka's most important design decisions. It determines which partition a record lands on — and since ordering is guaranteed only within a partition, the key is what controls the ordering of your events.

How keys map to partitions



The key is hashed to select a partition; the same key always maps to the same one.

By default, Kafka hashes the key and takes the result modulo the partition count. The same key therefore always maps to the same partition — and with no key, records are spread across partitions round-robin.

Key drives ordering

Because a key always lands on one partition, and a partition is strictly ordered, **all records with the same key are processed in order**. Choose the key to match your ordering requirement.

```java
// all events for one order stay ordered by keying on order id
producer.send(new ProducerRecord<>("order-events",
    order.getId(),           // key -> same partition -> ordered
    event));
```

Choosing a key

Goal	Key by
order per user	user id
order per order	order id
max spread, no order	no key (round-robin)

Watch for hot partitions

WATCH OUT

A key with skewed distribution — one very active user, say — sends most traffic to a single partition, creating a **hot partition** that limits throughput. Choose keys that spread load reasonably evenly.

The key controls partition and order

A record's key is hashed to pick a partition, so the same key always lands together and stays ordered. Choose the key to match your ordering needs — and avoid skewed keys that overload a single partition.

© 2026 ByteMart Digital

MODULE 2 · PRODUCERS

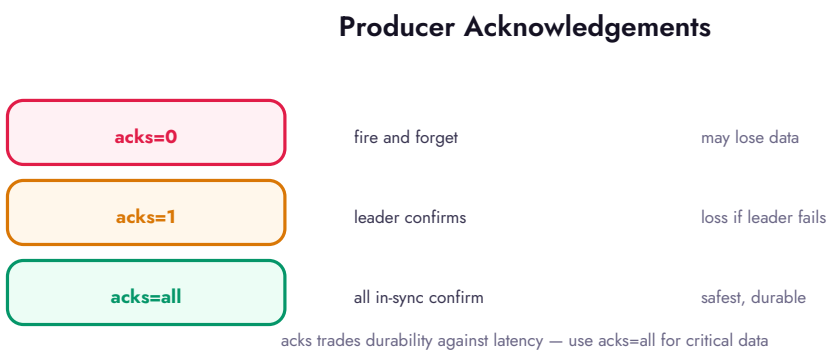
CHAPTER 07

Acknowledgements & Durability

The acks setting that trades durability against latency when producing records.

When a producer sends a record, how sure should it be that the record is safely stored? The `acks` setting answers this — the central trade-off between **durability** and **latency**. Getting it right is essential for data you cannot afford to lose.

The acks levels



acks=0, 1, or all — trading durability against latency.

acks	Waits for	Risk
0	nothing	may lose data
1	leader only	loss if leader fails
all	all in-sync replicas	safest

Configuring durability

For critical data, combine `acks=all` with a minimum number of in-sync replicas, so a write is confirmed only when safely replicated.

```java
props.put("acks", "all");                 // wait for all ISR
props.put("enable.idempotence", "true");  // no duplicates on retry
props.put("retries", Integer.MAX_VALUE);  // retry transient failures
// broker/topic side: min.insync.replicas = 2
```

The trade-off

- **acks=0** — fastest, no guarantee; fine for lossy metrics.
- **acks=1** — balanced, but loses data if the leader fails before replication.
- **acks=all** — strongest durability; slightly higher latency.
- Pair **acks=all** with `min.insync.replicas` for real safety.

© 2026 ByteMart Digital

WATCH OUT

`acks=all` alone is not enough — without `min.insync.replicas > 1`, a write can be 'fully' acknowledged by just the leader if replicas have fallen out of sync. Set both together for genuine durability.

acks trades safety for speed

The `acks` setting decides how many replicas must confirm a write: 0 (fastest, lossy), 1 (balanced), or all (durable). For data that matters, use `acks=all` with `min.insync.replicas` — and enable idempotence.

© 2026 ByteMart Digital

MODULE 2 · PRODUCERS

CHAPTER 08

Batching, Compression & Throughput

How producers achieve high throughput by batching and compressing records.

Kafka producers achieve enormous throughput by not sending records one at a time. Instead they **batch** many records per request and **compress** them, dramatically reducing network and storage cost. Tuning these settings is how you balance throughput against latency.

Batching

The producer accumulates records into per-partition batches and sends them together. Two settings control this: `batch.size` (how big a batch grows) and `linger.ms` (how long to wait for more records before sending).

```java
props.put("batch.size", 32768);      // 32 KB batches
props.put("linger.ms", 10);          // wait up to 10ms to fill a batch
props.put("compression.type", "lz4");  // compress each batch
```

The latency-throughput trade-off

A larger `linger.ms` lets batches fill more, improving throughput and compression — at the cost of a little latency. It is the main dial between the two.

Setting	Higher value means
batch.size	bigger batches, better throughput
linger.ms	more batching, more latency
compression.type	less network/disk, some CPU

Compression

- Compresses whole batches, so bigger batches compress better.
- **lz4** and **zstd** balance speed and ratio well.
- Cuts network transfer and disk storage significantly.
- Compressed data stays compressed on the broker and to consumers.

Tuning for throughput

TIP

For high-throughput pipelines, increase `batch.size` and `linger.ms` and enable compression — together they can multiply throughput several times over. For low-latency needs, keep `linger.ms` small.

© 2026 ByteMart Digital

Batch and compress for scale

Producers reach high throughput by batching records and compressing batches. `batch.size` and `linger.ms` tune the throughput-latency balance, and compression cuts network and storage cost — the core levers for producer performance.

© 2026 ByteMart Digital

MODULE 3 · CONSUMERS

CHAPTER 09

Consuming Messages

How a consumer subscribes to topics and continuously polls for records.

A **consumer** reads events from Kafka. It subscribes to one or more topics and continuously **polls** for new records, processing each batch it receives. The poll loop is the heart of every Kafka consumer, and understanding it explains much of a consumer's behaviour.

A basic consumer

java

```
Properties props = new Properties();
props.put("bootstrap.servers", "broker:9092");
props.put("group.id", "billing");
props.put("key.deserializer", StringDeserializer.class.getName());
props.put("value.deserializer", StringDeserializer.class.getName());

try (var consumer = new KafkaConsumer<String, String>(props)) {
    consumer.subscribe(List.of("orders"));
    while (true) {
        var records = consumer.poll(Duration.ofMillis(100));
        for (var record : records)
            process(record.key(), record.value());
    }
}
```

The poll loop

`poll()` does more than fetch records — it also sends heartbeats, handles group coordination, and triggers rebalances. A consumer must poll regularly, or the group will consider it dead.

- **Subscribe** to topics; Kafka assigns partitions to you.
- **Poll** in a loop to fetch batches of records.
- **Process** each record's key and value.
- Polling also drives **heartbeats** and coordination.

Keep the loop responsive

WATCH OUT

If processing a batch takes too long between polls, Kafka assumes the consumer has failed and rebalances its partitions away. Keep per-poll work bounded, or offload slow processing so you keep polling in time.

Pull, not push

Kafka consumers **pull** data at their own pace rather than having it pushed to them. This lets each consumer control its own throughput and naturally applies backpressure — a slow consumer simply polls less.

© 2026 ByteMart Digital

Subscribe and poll

A consumer subscribes to topics and polls in a loop to pull batches of records, with the poll call also driving heartbeats and coordination. Keep processing bounded so you poll in time, and let pull-based consumption pace itself.

MODULE 3 · CONSUMERS

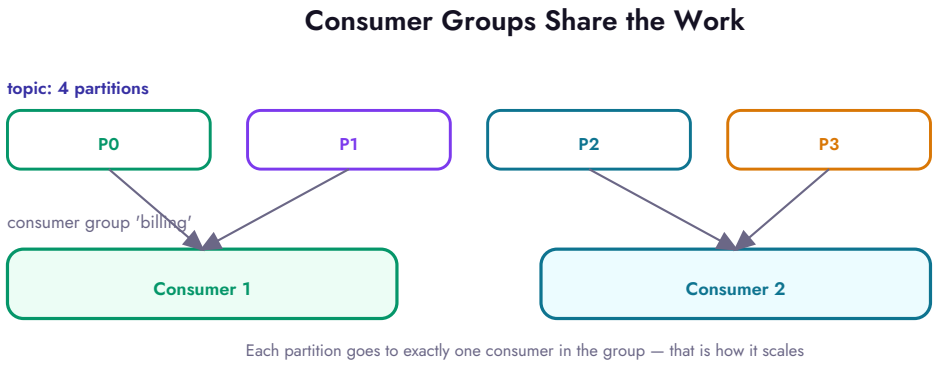
CHAPTER 10

Consumer Groups & Rebalancing

How consumers scale by sharing a topic's partitions across a group.

A single consumer can only read so fast. **Consumer groups** are how Kafka scales consumption: several consumers share a group id, and Kafka divides the topic's partitions among them — so they process in parallel without ever reading the same record twice.

How a group shares partitions



Kafka assigns each partition to exactly one consumer in a group, enabling parallel processing.

Kafka assigns each partition to **exactly one** consumer within a group. Multiple groups each get their own full copy of the data, but within a group, the partitions are divided up for parallelism.

The scaling rule

- Add consumers to a group to process more partitions in parallel.
- Parallelism is capped by the **partition count** — extra consumers idle.
- Each partition is owned by one consumer at a time.
- Different groups consume the same topic independently.

WATCH OUT

You cannot have more actively-consuming members than partitions — with 4 partitions and 6 consumers, 2 sit idle. Size the partition count for your target consumer parallelism, since it caps how far you can scale out.

Rebalancing

When a consumer joins or leaves the group (a deploy, a crash, a scale-up), Kafka **rebalances** — reassigning partitions among the current members. During a rebalance, processing briefly pauses.

Minimising rebalance impact

- **Cooperative rebalancing** reassigns only what must move, not everything.
- **Static membership** avoids rebalances on brief restarts.

© 2026 ByteMart Digital

- Tune session and heartbeat timeouts to your restart patterns.
- Frequent rebalances hurt throughput — keep the group stable.

Groups scale consumption

A consumer group divides a topic's partitions among its members for parallel processing, capped by the partition count. Rebalancing reassigns partitions when membership changes — keep groups stable and use cooperative rebalancing to minimise disruption.

© 2026 ByteMart Digital

MODULE 3 · CONSUMERS

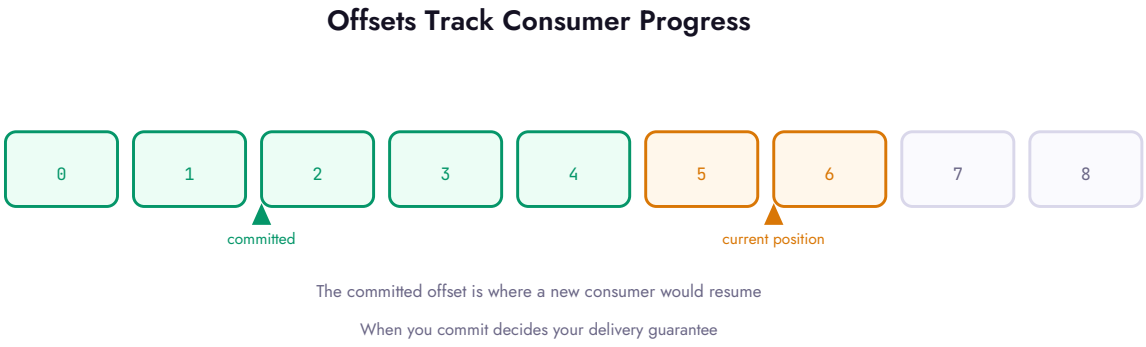
CHAPTER 11

Offsets & Commit Strategies

How consumers track progress by committing offsets — and why commit timing matters.

A consumer must remember what it has processed so it can resume after a restart or rebalance. It does this by **committing offsets** — recording its position in each partition. When you commit, relative to processing, determines your delivery guarantee.

What a committed offset means



The committed offset is where a new consumer resumes; the position is where it currently reads.

The **committed offset** is the position Kafka remembers for the group. If the consumer restarts, it resumes from there — so committing too early or too late changes what happens to records around a failure.

Auto vs manual commit

Mode	Behaviour
auto commit	commits periodically in the background
manual commit	you commit explicitly after processing

```java
props.put("enable.auto.commit", "false");   // take control

var records = consumer.poll(Duration.ofMillis(100));
for (var record : records) process(record);
consumer.commitSync();     // commit only after successful processing
```

Commit timing decides delivery

- Commit **before** processing — a crash loses records (at-most-once).
- Commit **after** processing — a crash reprocesses (at-least-once).
- Manual commit after processing is the safe, common choice.
- The next chapter covers these semantics in full.

© 2026 ByteMart Digital

WATCH OUT

Auto-commit is convenient but can commit records you have fetched but not yet processed — losing them on a crash. For anything important, disable auto-commit and commit manually after processing succeeds.

Commit reflects real progress

Committed offsets are how a consumer resumes after failure. Prefer manual commits after successful processing so a crash reprocesses rather than loses records — commit timing is precisely what sets your delivery guarantee.

© 2026 ByteMart Digital

MODULE 3 · CONSUMERS

CHAPTER 12

Consumer Delivery Semantics

The three delivery guarantees and how commit strategy and idempotency achieve them.

Every Kafka application must answer one question: what happens to a record if the consumer fails mid-processing? The answer is its **delivery semantics** — at-most-once, at-least-once, or exactly-once — determined by how you commit and how you process.

The three guarantees



At-most-once, at-least-once, and exactly-once — the three delivery guarantees.

Semantic	On failure
at most once	records may be lost
at least once	records may be duplicated
exactly once	no loss, no duplicates

At-least-once is the default

The most common choice: commit after processing. If the consumer crashes before committing, it reprocesses those records on restart — so nothing is lost, but some records may be handled twice.

Idempotency: making duplicates harmless

The practical way to live with at-least-once is **idempotent processing** — designing operations so that handling a record twice has the same effect as once. Then duplicates simply do not matter.

```java
// idempotent: upsert by a unique id, safe to repeat
repository.upsert(event.getId(), event.getData());
// re-processing the same event produces the same result
```

Exactly-once

- Achieved with Kafka **transactions** or idempotent processing.
- The read-process-write cycle commits atomically (next module).
- Stronger, but with overhead — not always necessary.
- Often, **at-least-once + idempotency** is simpler and enough.

Choose your guarantee deliberately

Delivery semantics — at-most-once, at-least-once, exactly-once — follow from your commit and processing strategy. At-least-once with idempotent processing is the pragmatic default; reserve true exactly-once for when duplicates genuinely cannot be tolerated.

© 2026 ByteMart Digital

MODULE 4 · RELIABILITY & REPLICATION

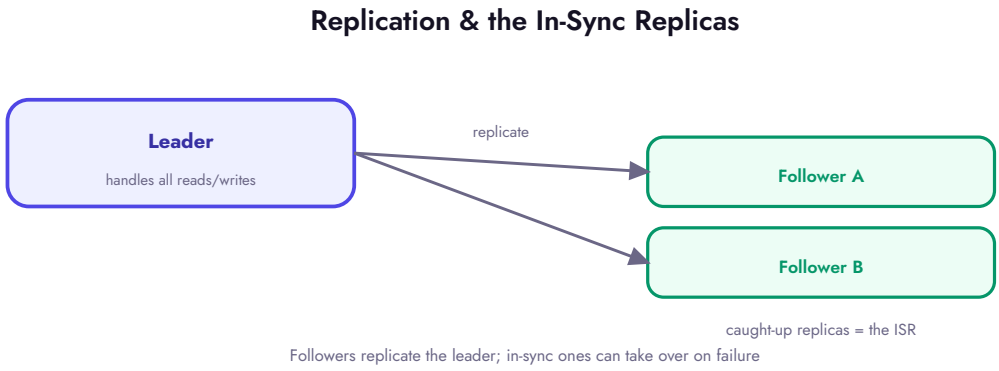
CHAPTER 13

Replication & the ISR

How Kafka replicates partitions across brokers and tracks which replicas are in sync.

Kafka survives broker failures by **replicating** every partition across multiple brokers. One replica leads; the others follow, copying its data. The set of replicas that are fully caught up — the **in-sync replicas** — is central to both durability and availability.

Leaders, followers, and the ISR



The leader handles reads and writes; in-sync followers replicate it and can take over.

Each partition has a **leader** replica that handles all reads and writes, and **follower** replicas that continuously copy from it. Followers that are sufficiently caught up form the **in-sync replica set (ISR)**.

The replication factor

The **replication factor** sets how many copies of each partition exist. A factor of 3 is the common production choice — it tolerates the loss of one broker with room to spare.

Factor	Tolerates
1	no broker failure (avoid in prod)
2	one broker failure
3	one failure comfortably (recommended)

Why the ISR matters

- Only **in-sync** replicas are eligible to become leader.
- `acks=all` waits for all ISR members to confirm a write.
- `min.insync.replicas` sets how many must be in sync to accept writes.
- A replica that falls behind is removed from the ISR until it catches up.

Durability comes from the ISR

Durability is the combination of replication factor, `acks=all`, and `min.insync.replicas`. Together they ensure a confirmed write survives broker failures — no single setting is enough alone.

Replicated, with an in-sync set

Kafka replicates each partition across brokers, with one leader and caught-up followers forming the ISR. Durability comes from combining a replication factor of 3, `acks=all`, and `min.insync.replicas` — the ISR is what makes writes survive failure.

© 2026 ByteMart Digital

MODULE 4 · RELIABILITY & REPLICATION

CHAPTER 14

Leaders, Followers & Failover

How Kafka elects partition leaders and fails over automatically when a broker dies.

When the broker leading a partition fails, Kafka must promote a follower to take its place — automatically and without losing acknowledged data. This **failover** is what makes the cluster resilient, and understanding it explains how availability and durability interact.

Leader election

Each partition's leader handles all its traffic. If that broker fails, the cluster controller elects a new leader from the partition's **in-sync replicas** — a replica guaranteed to have the acknowledged data.

- Only **in-sync** replicas are normally eligible for election.
- The controller detects the failure and promotes a new leader.
- Clients discover the new leader and reconnect automatically.
- Acknowledged writes are preserved because the ISR had them.

The availability vs durability trade-off

What if no in-sync replica is available? **Unclean leader election** decides: allow an out-of-sync replica to become leader (available, but may lose data) or wait for an in-sync one to return (durable, but unavailable meanwhile).

unclean.leader.election	Result
disabled (default)	durable; may pause on failure
enabled	available; may lose data

WATCH OUT

Leave **unclean leader election disabled** for data you cannot lose — the default. Enabling it trades durability for availability, letting a stale replica become leader and silently drop acknowledged records.

Spreading risk with racks

Rack awareness places a partition's replicas across different racks or availability zones, so a whole-rack outage cannot take down every replica of a partition at once.

Automatic, in-sync failover

When a leader fails, Kafka promotes an in-sync replica automatically, preserving acknowledged writes; clients reconnect transparently. Keep unclean leader election off for durability, and use rack awareness to survive zone failures.

© 2026 ByteMart Digital

MODULE 4 · RELIABILITY & REPLICATION

CHAPTER 15

Delivery Guarantees End to End

Combining producer and consumer settings to achieve a guarantee across the whole pipeline.

Delivery guarantees are not a single setting — they are an end-to-end property. A durable producer paired with a careless consumer still loses data. This chapter brings the producer and consumer sides together to reason about the guarantee your whole pipeline provides.

Both sides must agree

The overall guarantee is the weaker of the two ends. To achieve at-least-once end to end, both the write and the read must be at-least-once.

Side	For at-least-once
Producer	acks=all, retries, idempotence
Broker	replication + min.insync.replicas
Consumer	commit after processing

The durable producer

```java
props.put("acks", "all");
props.put("enable.idempotence", "true");   // no duplicates on retry
props.put("retries", Integer.MAX_VALUE);
```

The safe consumer

```java
props.put("enable.auto.commit", "false");
// process, THEN commit
for (var record : records) process(record);
consumer.commitSync();
```

Reasoning about the pipeline

- A durable producer with a lossy consumer still loses data.
- The end-to-end guarantee is the **weakest link**.
- At-least-once + idempotent processing is the robust default.
- True exactly-once needs transactions across the read-process-write cycle.

© 2026 ByteMart Digital

WATCH OUT

Do not assume `acks=all` alone gives reliable delivery. If the consumer commits before processing, records are still lost on a crash — the guarantee is only as strong as the weakest end of the pipeline.

Guarantees are end to end

A delivery guarantee spans producer, broker, and consumer — and equals the weakest link. Combine a durable producer (`acks=all`, idempotence), replicated brokers, and a consumer that commits after processing to make at-least-once hold across the whole pipeline.

© 2026 ByteMart Digital

MODULE 4 · RELIABILITY & REPLICATION

CHAPTER 16

Idempotent & Transactional Producers

Eliminating duplicates and achieving exactly-once with idempotent and transactional producers.

Two producer features push Kafka toward the strongest guarantees. The **idempotent producer** eliminates duplicates from retries, and **transactions** let a producer write to multiple partitions atomically — together enabling true exactly-once processing.

The idempotent producer

Without it, a producer retry after a network glitch can write the same record twice. Enabling idempotence makes Kafka detect and discard these duplicates, so retries are safe.

java

```
props.put("enable.idempotence", "true");
// Kafka assigns sequence numbers so duplicate writes are dropped;
// acks=all and safe retries are configured automatically
```

Transactions

A **transaction** groups multiple sends — across partitions and topics — so they all commit or all abort atomically. This is the key to the exactly-once **read-process-write** pattern in stream processing.

java

```
producer.initTransactions();
try {
    producer.beginTransaction();
    producer.send(record1);
    producer.send(record2);
    producer.sendOffsetsToTransaction(offsets, groupMetadata);
    producer.commitTransaction();   // all-or-nothing
} catch (Exception e) {
    producer.abortTransaction();
}
```

Exactly-once semantics (EOS)

- Idempotence removes duplicates from producer retries.
- Transactions make multi-partition writes atomic.
- Committing consumer offsets within the transaction ties read to write.
- Kafka Streams enables exactly-once with a single config setting.

When you need it

TIP

Enable idempotence always — it is nearly free and prevents retry duplicates. Reserve full transactions for genuine exactly-once needs like financial processing; for many pipelines, at-least-once plus idempotent handling is simpler and sufficient.

Toward exactly-once

The idempotent producer removes retry duplicates, and transactions make multi-partition writes atomic — together enabling exactly-once processing. Always enable idempotence; add transactions when your pipeline truly cannot tolerate duplicates or partial writes.

© 2026 ByteMart Digital

MODULE 5 · DATA & SCHEMAS

CHAPTER 17

Serialization & Formats

How records are turned into bytes, and choosing a serialization format for your events.

Kafka stores and transports raw bytes — it neither knows nor cares what your records mean. **Serialization** turns your objects into bytes to produce, and back into objects to consume. The format you choose shapes efficiency, evolvability, and how well producers and consumers stay compatible.

Serializers and deserializers

A producer needs a **serializer** for keys and values; a consumer needs matching **deserializers**. They must agree on the format, or the consumer gets garbage.

```java
// producer side
props.put("value.serializer", "...KafkaAvroSerializer");
// consumer side must match
props.put("value.deserializer", "...KafkaAvroDeserializer");
```

Format choices

Format	Character
JSON	human-readable, verbose, no schema
Avro	compact binary, schema-based
Protobuf	compact binary, schema-based
String / bytes	simple, unstructured

Why schema-based formats win at scale

- **Compact** — binary encoding is far smaller than JSON.
- **Typed** — a schema documents and enforces the structure.
- **Evolvable** — schemas can change under compatibility rules.
- **Safe** — the registry catches breaking changes before they ship.

The cost of schemaless JSON

WATCH OUT

Plain JSON is easy to start with but has no enforced contract — a producer can silently change a field and break every consumer. For anything beyond prototypes, prefer a schema-based format with a registry.

Bytes need an agreed format

Kafka moves bytes, so producers and consumers must agree on serialization. Schema-based formats like Avro and Protobuf are compact, typed, and evolvable — far safer at scale than schemaless JSON, which offers no contract between teams.

© 2026 ByteMart Digital

MODULE 5 · DATA & SCHEMAS

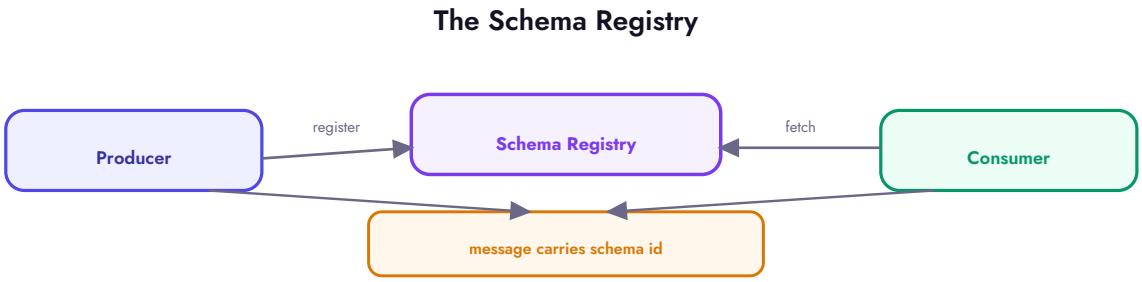
CHAPTER 18

The Schema Registry

Centralising schemas so producers and consumers share a versioned contract.

When events carry structured data, producers and consumers need a shared, versioned contract. The **Schema Registry** provides exactly this: a central store of schemas that both sides reference, so messages carry only a small schema id instead of the full schema.

How the registry works



Schemas live centrally; producers register them and messages carry a compact schema id.

- The producer **registers** its schema and gets a schema id.
- Each message carries the small **id**, not the whole schema.
- The consumer **fetches** the schema by id to deserialize.
- Schemas are versioned and stored centrally.

Why it matters

The registry turns the data format into an enforced, shared contract. Producers cannot register an incompatible schema (under the configured rules), so consumers are protected from breaking changes — and messages stay compact.

Subjects and versions

Concept	Is
Subject	a named scope for a schema (per topic)
Version	one registered revision of a schema
Schema id	a global id embedded in each message

Using it in practice

```java
props.put("schema.registry.url", "http://registry:8081");
props.put("value.serializer",
    "io.confluent.kafka.serializers.KafkaAvroSerializer");
// the serializer registers/fetches schemas automatically
```

A shared, versioned contract

The Schema Registry centralises schemas so producers and consumers agree on structure, with messages carrying just a compact schema id. It enforces compatibility and keeps messages small — the backbone of safe data exchange between teams.

MODULE 5 · DATA & SCHEMAS

CHAPTER 19

Schema Evolution & Compatibility

Changing event schemas over time without breaking existing producers or consumers.

Data structures change — you add a field, deprecate another. In a system where many independent producers and consumers share a topic, changing a schema carelessly breaks everyone. **Compatibility rules** let schemas evolve safely, so old and new sides keep working together.

Compatibility types

Type	Guarantees
BACKWARD	new consumers read old data
FORWARD	old consumers read new data
FULL	both directions work
NONE	no checks (avoid)

Safe evolution rules

- **Adding** a field with a default is safe.
- **Removing** a field with a default is safe (backward).
- Never **rename** or change the **type** of a field.
- Never remove a required field with no default.

Why backward compatibility is common

BACKWARD compatibility — new consumers can read data written with the old schema — is the usual default. It lets you deploy updated consumers first, then producers, without a coordinated big-bang release.

```text
// Avro: adding a field safely (has a default)
{ "name": "discount", "type": "int", "default": 0 }
// old messages without 'discount' still deserialize -> 0
```

Enforcement

WATCH OUT

Set a compatibility level on each subject and let the registry **reject** incompatible schemas at registration — catching a breaking change in CI, not in production when consumers start failing to deserialize.

© 2026 ByteMart Digital

Evolve schemas safely

Compatibility rules — backward, forward, full — let schemas change without breaking producers or consumers. Add fields with defaults, never rename or retype, and let the registry reject incompatible changes so evolution never becomes an outage.

© 2026 ByteMart Digital

MODULE 6 · STREAM PROCESSING

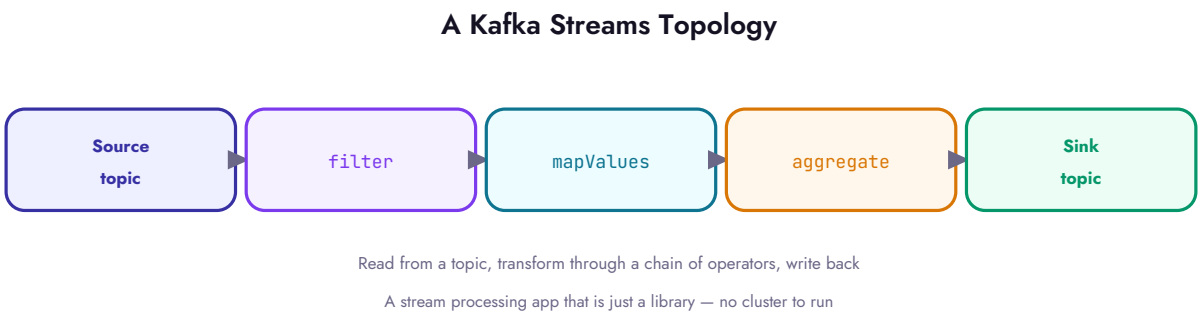
CHAPTER 20

Introduction to Kafka Streams

Processing streams of events in real time with a library that runs inside your application.

Reading, transforming, and re-publishing events is so common that Kafka provides a library for it: **Kafka Streams**. It lets you build real-time processing applications — filtering, transforming, aggregating, joining — as an ordinary Java app, with no separate cluster to run.

A processing topology



A Streams app reads from a topic, transforms through operators, and writes back.

A Streams application defines a **topology** — a graph of processing steps from source topics through operators to sink topics. Kafka Streams runs this topology, handling scaling, state, and fault tolerance.

A simple stream

```java
StreamsBuilder builder = new StreamsBuilder();
builder.stream("orders", Consumed.with(Serdes.String(), orderSerde))
    .filter((k, order) -> order.total() > 1000)
    .mapValues(Order::toSummary)
    .to("high-value-orders");

var streams = new KafkaStreams(builder.build(), props);
streams.start();
```

Why Kafka Streams

- **Just a library** — no cluster; runs inside your service.
- **Scalable** — runs many instances that share the work, like a consumer group.
- **Fault-tolerant** — state is backed up to Kafka and restored on failure.
- **Exactly-once** — available with a single configuration setting.

Streams vs a plain consumer

You could do this with a consumer and producer by hand — but Streams gives you stateful operations, windowing, joins, and fault-tolerant local state for free. It is the right tool whenever processing is more than trivial.

Real-time processing as a library
Kafka Streams builds real-time processing topologies — filter, transform, aggregate, join — as a plain library inside your app, with no cluster to manage. It scales like a consumer group and handles state and fault tolerance for you.

© 2026 ByteMart Digital

MODULE 6 · STREAM PROCESSING

CHAPTER 21

Stateless Operations

The transformations that process each record independently — map, filter, branch, and more.

The simplest stream operations are **stateless**: each record is processed on its own, with no memory of others. Filtering, mapping, and branching cover a huge share of real processing needs, and because they hold no state, they are simple and cheap.

Core stateless operations

Operation	Does
filter	keep records matching a predicate
map / mapValues	transform each record
flatMap	produce zero or more records each
branch / split	route records to different streams
foreach	a terminal side effect per record

Transforming a stream

java

```
KStream<String, Order> orders = builder.stream("orders");

orders
    .filter((k, o) -> o.isPaid())
    .mapValues(o -> enrich(o))          // transform value
    .selectKey((k, o) -> o.customerId()) // re-key
    .to("enriched-orders");
```

Re-keying and repartitioning

Changing the key with `selectKey` or `map` means records may need to move to different partitions. Kafka Streams handles this **repartitioning** automatically, writing to an internal topic when required.

- **filter / filterNot** — drop records by a condition.
- **mapValues** — transform the value, keeping the key (no repartition).
- **map / selectKey** — change the key (may repartition).
- **flatMap** — split one record into many, or none.

TIP

Prefer `mapValues` over `map` when you only change the value — it preserves the key and avoids an unnecessary repartition, which costs an extra internal topic and network round trip.

Process each record alone

Stateless operations — filter, map, flatMap, branch — transform records independently and cover most processing needs cheaply. Prefer value-only transforms to avoid repartitioning, and re-key only when downstream grouping or joins require it.

© 2026 ByteMart Digital

MODULE 6 · STREAM PROCESSING

CHAPTER 22

Stateful Operations & Windowing

Aggregating and counting across records over time, backed by fault-tolerant local state.

Real analytics need **memory**: counts, sums, running totals across many records. These **stateful** operations maintain local state — backed up to Kafka for fault tolerance — and **windowing** bounds them in time, so you can compute things like 'orders per minute'.

Aggregations

Grouping records by key and then aggregating produces a running result per key. Kafka Streams keeps this state locally and mirrors it to a compacted **changelog topic** so it survives failures.

```java
orders
    .groupBy((k, o) -> o.customerId())
    .aggregate(
        () -> 0L,                          // initial value
        (custId, order, total) -> total + order.total())
    .toStream()
    .to("customer-totals");
```

Windowing

A **window** groups records within a time bound, so an aggregation resets per window. This turns 'total per customer' into 'total per customer per hour'.

Window	Groups by
Tumbling	fixed, non-overlapping intervals
Hopping	fixed size, overlapping
Session	activity separated by gaps
Sliding	pairs within a time difference

```java
orders
    .groupByKey()
    .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(1)))
    .count()      // count per key, per one-minute window
    .toStream()
    .to("orders-per-minute");
```

State and fault tolerance

- State lives in a fast local store (RocksDB) on each instance.

- It is backed by a **changelog** topic in Kafka.
- On failure or rebalance, state is **restored** from the changelog.
- Late events are handled with a configurable **grace period**.

Aggregate over time, safely

Stateful operations — aggregate, count, reduce — maintain running results per key, and windowing bounds them in time. Local state backed by a changelog topic makes this fault-tolerant, so real-time analytics survive failures and rebalances.

© 2026 ByteMart Digital

MODULE 6 · STREAM PROCESSING

CHAPTER 23

KStreams, KTables & Joins

The two views of stream data and how joining them enables real-time enrichment.

Kafka Streams offers two ways to see data: a **KStream** is a sequence of independent events, while a **KTable** is a changing table where each key has a current value. Understanding this duality — and how to **join** them — unlocks powerful real-time enrichment.

Stream-table duality

	KStream	KTable
Represents	a flow of events	current state per key
Each record is	a new fact	an update to a key
Think	insert-only log	latest-value map

A stream of updates is a table (apply each update), and a table's changes are a stream. This duality is the conceptual heart of stream processing.

Joins

Joining lets you enrich events with related data in real time — combining an order stream with a customer table, for instance. Kafka Streams supports several join types across streams and tables.

Join	Combines
Stream-Table	enrich events with current state
Stream-Stream	correlate two event flows (windowed)
Table-Table	join two changing tables

```java
KStream<String, Order> orders = builder.stream("orders");
KTable<String, Customer> customers = builder.table("customers");

orders
    .join(customers, (order, customer) ->
        new EnrichedOrder(order, customer.name(), customer.tier()))
    .to("enriched-orders");   // each order enriched with customer data
```

When to use each

- **KStream** — events where every record matters (clicks, transactions).
- **KTable** — latest state per key (profiles, inventory, prices).
- **Stream-table join** — the workhorse for real-time enrichment.
- **Stream-stream join** — correlating events within a time window.

© 2026 ByteMart Digital

Two views, joined in real time

A KStream is a flow of events; a KTable is current state per key — two sides of the same data. Joining them, especially stream-table joins, enriches events with live state in real time, which is one of stream processing's most valuable capabilities.

MODULE 7 · INTEGRATION & ECOSYSTEM

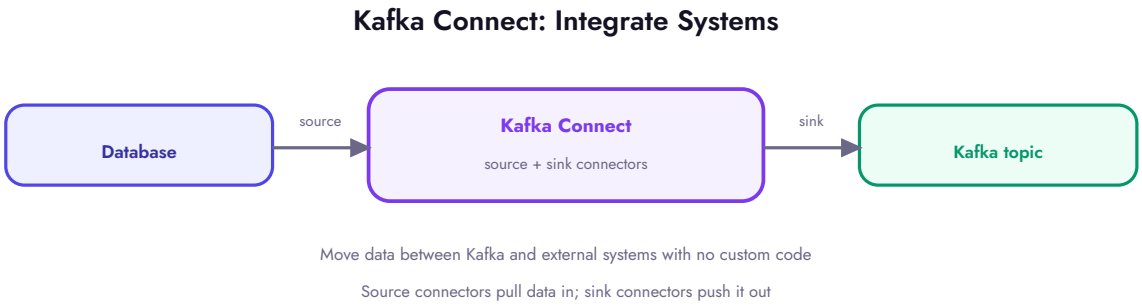
CHAPTER 24

Kafka Connect

Moving data between Kafka and external systems without writing custom integration code.

Most Kafka use cases involve getting data into Kafka from databases and systems, and out to warehouses and search indexes. **Kafka Connect** does this with configuration instead of code — a framework of ready-made **connectors** for common systems.

Source and sink connectors



Source connectors pull data into Kafka; sink connectors push it out to other systems.

Connector	Direction
Source	external system -> Kafka
Sink	Kafka -> external system

Configuration, not code

You configure a connector with a small JSON document — no integration code to write, test, or maintain. Connect handles the scaling, offset tracking, and fault tolerance.

```text
{
  "name": "orders-source",
  "config": {
    "connector.class": "io.debezium.connector.postgresql...",
    "database.hostname": "db",
    "table.include.list": "public.orders",
    "topic.prefix": "cdc"
  }
}
```

Change data capture

A powerful pattern: **change data capture (CDC)** connectors (like Debezium) stream every insert, update, and delete from a database into Kafka — turning your database changelog into an event stream, with no application

changes.

Why use Connect

- Ready-made connectors for databases, storage, search, and more.
- **No custom code** — just configuration.
- Runs distributed, with scaling and fault tolerance built in.
- Handles offsets and delivery guarantees for you.

Integration by configuration

Kafka Connect moves data between Kafka and external systems using configurable source and sink connectors — no integration code required. It handles scaling and fault tolerance, and CDC connectors turn a database's changes into an event stream.

MODULE 7 · INTEGRATION & ECOSYSTEM

CHAPTER 25

Connectors & Transformations

Running connectors in practice and reshaping data in flight with single message transforms.

Beyond picking a connector, running Connect well means understanding its distributed model and how to reshape data as it flows. **Single Message Transforms** let you tweak records in flight — renaming fields, masking data, routing topics — without any code.

Distributed Connect

Connect runs as a cluster of **workers** that share connector **tasks**. Tasks are distributed across workers for scale, and if a worker fails, its tasks are reassigned — just like a consumer group.

- A **connector** defines the integration; it splits into **tasks**.
- Tasks run across a cluster of **workers** for parallelism.
- Connect tracks source and sink **offsets** automatically.
- Failed tasks are rebalanced onto healthy workers.

Single Message Transforms (SMTs)

An **SMT** modifies each record as it passes through Connect — lightweight, configurable, and chainable. Use them for the small reshaping that would otherwise need a custom processor.

text

```
"transforms": "mask,route",
"transforms.mask.type": "...MaskField$Value",
"transforms.mask.fields": "ssn,card",
"transforms.route.type": "...RegexRouter",
"transforms.route.regex": "(.*)",
"transforms.route.replacement": "prod.$1"
```

Common SMTs

SMT	Does
MaskField	hide sensitive fields
ReplaceField	rename or drop fields
RegexRouter	rewrite the target topic
InsertField	add metadata fields

WATCH OUT

SMTs are for **simple, per-record** tweaks — renaming, masking, routing. For real transformation, aggregation, or joins, use Kafka Streams instead; overloading connectors with complex logic makes pipelines hard to understand and maintain.

© 2026 ByteMart Digital

Run connectors, reshape in flight

Connect runs connectors as distributed, fault-tolerant tasks across workers, and Single Message Transforms reshape records in flight without code. Use SMTs for simple per-record tweaks, and reach for Kafka Streams when the logic gets real.

© 2026 ByteMart Digital

MODULE 7 · INTEGRATION & ECOSYSTEM

CHAPTER 26

Kafka with Spring

Building Kafka producers, consumers, and streams idiomatically with Spring for Apache Kafka.

In the Spring ecosystem, **Spring for Apache Kafka** makes producing and consuming idiomatic and concise. It wraps the native clients with familiar Spring abstractions — a template for sending, annotations for listening — and integrates cleanly with Spring Boot's configuration.

Configuration

Spring Boot auto-configures Kafka from properties — bootstrap servers, serializers, and consumer group all live in `application.yml`.

```text
spring:
  kafka:
    bootstrap-servers: broker:9092
    consumer:
      group-id: billing
      auto-offset-reset: earliest
    producer:
      acks: all
```

Producing with KafkaTemplate

```java
@Service
public class OrderPublisher {
    private final KafkaTemplate<String, Order> template;
    public OrderPublisher(KafkaTemplate<String, Order> template) {
        this.template = template;
    }
    public void publish(Order order) {
        template.send("orders", order.getId(), order);
    }
}
```

Consuming with @KafkaListener

A single annotation turns a method into a consumer. Spring manages the poll loop, deserialization, error handling, and offset commits for you.

```java
@KafkaListener(topics = "orders", groupId = "billing")
public void onOrder(Order order) {
    billingService.charge(order);   // Spring handles poll + commit
}
```

© 2026 ByteMart Digital

What Spring adds

- **KafkaTemplate** — a clean send API with callbacks.
- **@KafkaListener** — declarative consumers, no poll loop.
- **Error handling** — retries and dead-letter topics with config.
- **Transactions** and Kafka Streams support out of the box.

Kafka the Spring way

Spring for Apache Kafka wraps the native clients with a template for producing and `@KafkaListener` for consuming, auto-configured by Spring Boot. It removes boilerplate — poll loops, commits, error handling — letting you focus on the event logic.

© 2026 ByteMart Digital

MODULE 7 · INTEGRATION & ECOSYSTEM

CHAPTER 27

Event-Driven Microservices

Using Kafka as the backbone for loosely-coupled, event-driven microservices.

Kafka's most transformative use is as the **backbone of event-driven microservices**. Instead of services calling each other directly, they publish events to Kafka and react to events they care about — achieving the loose coupling and resilience that direct calls cannot.

Events over direct calls

In an event-driven design, a service announces facts (`OrderPlaced`) rather than commanding others. Interested services subscribe and react. The producer neither knows nor cares who consumes.

- **Loose coupling** — services depend on events, not each other.
- **Resilience** — a consumer can be down; events wait in the log.
- **Extensibility** — add a new consumer without touching producers.
- **Replayability** — a new service can replay history to build its state.

The event-carried state transfer pattern

Events can carry enough data for consumers to maintain their own local view, avoiding synchronous callbacks to the source service. Each service keeps just the data it needs, updated by events.

Kafka and the saga pattern

Kafka is a natural fit for **choreographed sagas** — multi-service business processes where each service reacts to an event, does its work, and emits the next event, with compensating events on failure.

```text
OrderPlaced -> [Payment] -> PaymentTaken
            -> [Inventory] -> StockReserved
            -> [Shipping] -> Shipped
// a failure emits a compensating event to undo prior steps
```

WATCH OUT

Event-driven systems trade immediate consistency for decoupling — consumers see changes slightly later, and you must design for eventual consistency and idempotent handling. It is a powerful model, but not a drop-in replacement for every synchronous call.

Events as the backbone

Kafka lets microservices communicate through events instead of direct calls, delivering loose coupling, resilience, extensibility, and replay. It underpins patterns like event-carried state transfer and choreographed sagas — at the cost of embracing eventual consistency.

MODULE 8 · OPERATIONS & PRACTICE

CHAPTER 28

Monitoring & Performance

The metrics that reveal cluster health, and the levers that tune Kafka's performance.

Running Kafka in production means watching the right signals and knowing which levers to pull. A handful of metrics reveal the health of the cluster and your applications, and a handful of settings tune the balance between throughput, latency, and durability.

Key metrics to watch

Metric	Tells you
Consumer lag	how far behind consumers are
Under-replicated partitions	replication is unhealthy
Request latency	broker responsiveness
Broker disk / CPU	resource pressure

Consumer lag — the vital sign

Consumer lag — the gap between the latest offset and a group's committed offset — is the single most important application metric. Growing lag means consumers cannot keep up; watch and alert on it closely.

Under-replicated partitions

WATCH OUT

A non-zero count of **under-replicated partitions** means some replicas have fallen out of sync — your durability guarantee is degraded. Treat a persistent non-zero value as an alert, not a curiosity.

Performance levers

- **Partitions** — more allow more consumer parallelism.
- **Batching / compression** — producer throughput.
- **acks and replication** — durability vs latency.
- **Broker resources** — disk throughput is often the bottleneck.

Capacity planning

Size a cluster around throughput, retention, and replication: total storage is roughly data rate times retention times replication factor. Plan headroom so a broker failure does not tip the cluster over its limits.

© 2026 ByteMart Digital

Watch lag, tune deliberately

Monitor consumer lag and under-replicated partitions above all — they reveal whether consumers keep up and whether durability holds. Tune partitions, batching, and acks knowingly, and plan capacity around throughput, retention, and replication.

© 2026 ByteMart Digital

MODULE 8 · OPERATIONS & PRACTICE

CHAPTER 29

Security

Securing a Kafka cluster with encryption, authentication, and authorization.

A Kafka cluster often carries an organisation's most sensitive event data, so securing it is essential. Kafka's security rests on three pillars: **encryption** in transit, **authentication** of clients, and **authorization** of what they may do.

The three pillars

Pillar	Provided by
Encryption	TLS on the wire
Authentication	TLS certs or SASL
Authorization	ACLs per resource

Encryption in transit

TLS encrypts traffic between clients and brokers (and between brokers), preventing eavesdropping on sensitive event data as it crosses the network.

```text
security.protocol=SSL
ssl.truststore.location=/etc/kafka/truststore.jks
# brokers present certificates; traffic is encrypted
```

Authentication

Kafka authenticates clients via **TLS client certificates** or **SASL** mechanisms (SCRAM, OAUTHBEARER, Kerberos). Every client proves its identity before it can produce or consume.

Authorization with ACLs

Once authenticated, a client is authorized by **ACLs** — rules granting a principal specific operations (read, write) on specific resources (topics, groups). Apply least privilege: grant only what each service needs.

```text
# allow the billing service to read the orders topic
kafka-acls --add \
  --allow-principal User:billing \
  --operation Read --topic orders --group billing
```

- Encrypt all traffic with **TLS** — never run plaintext in production.
- Authenticate every client with certs or **SASL**.

© 2026 ByteMart Digital

- Grant least-privilege **ACLs** per service.
- Rotate credentials and audit access regularly.

Encrypt, authenticate, authorize

Kafka security combines TLS encryption, client authentication via certificates or SASL, and ACL-based authorization. Encrypt all traffic, verify every client, and grant least-privilege access per resource — event data is often the most sensitive you hold.

© 2026 ByteMart Digital

MODULE 8 · OPERATIONS & PRACTICE

CHAPTER 30

Best Practices & Anti-Patterns

The habits that make Kafka systems reliable, and the mistakes that undermine them.

Kafka is powerful but unforgiving of misuse. This closing chapter gathers the practices that lead to reliable, scalable systems — and the anti-patterns that quietly undermine them — distilled from everything in this guide.

Design best practices

- Choose **keys** to match your ordering needs and spread load evenly.
- Set **partition counts** for future parallelism — hard to reduce later.
- Use a **schema registry** with compatibility rules from day one.
- Prefer **at-least-once + idempotent** processing; add EOS only when needed.
- Replicate with factor **3** and `min.insync.replicas=2` for durability.

Operational best practices

- Monitor **consumer lag** and **under-replicated partitions**.
- Keep consumer groups **stable** to avoid frequent rebalances.
- Secure everything: **TLS**, authentication, and least-privilege ACLs.
- Plan capacity around throughput, retention, and replication.

Anti-patterns to avoid

Anti-pattern	Why it hurts
huge messages	Kafka is for small events, not blobs
too many tiny topics	operational and metadata overhead
skewed keys	hot partitions, uneven load
no schema governance	silent breaking changes
ignoring consumer lag	consumers silently fall behind

Treat the log as a contract

Above all, remember that a Kafka topic is a shared, long-lived contract between many teams. Evolve it carefully, govern its schemas, and design for eventual consistency — and Kafka becomes the reliable backbone of your entire event-driven architecture.

© 2026 ByteMart Digital

Discipline makes Kafka reliable

Reliable Kafka systems come from good keys and partitioning, schema governance, sensible delivery semantics, strong replication, and vigilant monitoring. Avoid huge messages, skewed keys, and ungoverned schemas — and treat every topic as a durable contract between teams.

© 2026 ByteMart Digital